

Listing SaaS — Conversation & Project History

A documented record of the conversation that produced the **Listing SaaS** project — from the original freelance business idea to a working, deployed GitHub repository. This file preserves the full context of what was requested, what was built, and the decisions made along the way.

Table of contents

1. [The original idea](#)
 2. [Asking for a SaaS](#)
 3. [Refining the requirements](#)
 4. [Where to run the commands](#)
 5. [Making the repo](#)
 6. [Pushing to GitHub](#)
 7. [This documentation](#)
-

1. The original idea

User — #2 — Product Listing (Arabic & Global)

Customer DM:

"Hey, can you add 50 products to my Shopify/Salla store?"

My response:

"Yes. Send me the product list (Excel) with images and specs."

Customer sends:

[Google Sheet with 50 products, supplier links, images]

My workflow:

- Open Shopify/Salla dashboard (2 min)
- For each product: upload image, paste title, write description, set price, add tags (4-5 min per product)
- Review 5 random products for quality (5 min)
- Send progress report (3 min)

Delivery:

[50 products live in store, organized by category]

Price: \$25-50 per 50 products (\$0.50-\$1 per product)

Recurrence: Weekly (sellers add new products regularly)

Revenue scenarios:

- Conservative: 2 orders/day × \$35 × 26 days = \$1,820/month (104 hours)
- Target: 5 orders/day × \$35 × 26 days = \$4,550/month (260 hours)
- Strong: 10 orders/day × \$35 × 26 days = \$9,100/month (520 hours — unrealistic solo)

Revenue per delivery hour: ~\$17-20/hour

What could make this fail:

- Customers expect 500 products for same price
- Platform changes (Salla/Shopify updates) slow down workflow
- Bulk upload tools make DIY easier
- Acquisition requires being on Mostaq/Fiverr (platform dependency)

yes / no ?

Assistant response

Quick answer: **Yes, I can help with most of this — but with important limits.**

What I can actually do for a single delivery like this:

- Read the customer's Google Sheet (50 products, specs, supplier links, image URLs)
- Download/prepare product images
- Generate Arabic + English titles/descriptions/tags from the specs
- Build a Shopify CSV import file (or use the Salla/Shopify REST/Admin API) so all 50 products go live organized by category
- Spot-check 5 random products and produce a progress report

What I can't do:

- I can't own the recurring business (take payments, message customers on Instagram/WhatsApp, be "on Mostaq/Fiverr" to win orders). That's the acquisition side — that's on you.
- I can't run weekly on autopilot unless you set up an automation (cron/OpenHands automation) and give it store API credentials + a steady input sheet each cycle.
- I won't log into a customer's Shopify/Salla dashboard with their password. We'd use a store API token/CSV instead — safer and faster (the 4-5 min/product manual step becomes near-instant, which kills your "platform dependency" risk and the "bulk upload tools make DIY easier" risk too, since you'd be the one with the automated pipeline).

So if your real question is "can OpenHands execute the product-listing step of this offer?" → Yes. Send me the Google Sheet + Shopify/Salla store (API key or admin CSV access), and I'll list the 50 products.

If your question is "is this business model viable as described?" → Yes, but the bottleneck is customer acquisition (Mostaq/Fiverr/DMs), not the fulfillment.

Fulfillment is exactly the part I can automate, so your real risk is getting 2-10 orders/day, not doing the work.

2. Asking for a SaaS

User

can u make a saas where i upload the sheet after acquisition the customers and the saas handle the rest even if it's using u note i need it on laptop and phone or whatsapp/telegram (so if i don't have my laptop)

Assistant response

Short answer: **Yes, this is very buildable.**

What the SaaS would do (end-to-end):

1. You (or your customer) sends a Google Sheet link — either pasted in a web dashboard, or sent to a WhatsApp/Telegram bot.
2. Backend reads the sheet (50 products: title, specs, price, image URLs, category).
3. For each product: generates Arabic + English title/description/tags, downloads/prepares images, pushes to the customer's Shopify or Salla store via their API.
4. Progress + QA report sent back to you (web + WhatsApp/Telegram).
5. You see the result on phone or laptop — no laptop needed if using the bot.

Architecture (simple, cheap, works on phone + laptop):

- Backend: FastAPI (Python)
- Frontend: lightweight mobile-friendly web dashboard

- Messaging: Telegram bot first (free, easiest); WhatsApp as phase 2 (needs WhatsApp Business API/Twilio)
- Store APIs: Shopify Admin API + Salla API
- The "brain": OpenHands Cloud automations or a direct LLM API call (cheaper)
- DB: Postgres or SQLite to start
- Hosting: one small VPS / Render / Railway (~\$5-20/mo)

Decisions to make before building:

1. Shopify, Salla, or both?
2. Telegram first, or WhatsApp too from day 1?
3. Multi-customer (real SaaS with logins) or just for you?
4. Where should the LLM calls run — your own OpenAI/Anthropic key, or via OpenHands?

Recommended phased approach:

- Phase 1 (ship this week, solo + Telegram): FastAPI + simple mobile web dashboard + Telegram bot. One Shopify store connection. Paste sheet → products go live → report back. No laptop needed.
- Phase 2: add Salla, add WhatsApp, add multi-customer logins + billing.
- Phase 3: let customers self-serve → real hands-off SaaS.

3. Refining the requirements

User

both Shopify, Salla maybe ask that in the bot and have a toggle in the saas to choose also in each new chat ask for the api unless it's in the profile so i can handle many or only my own
telegram first
for now it's mine first
if it can be OpenHands for free then that else openrouter/free

Assistant action

All clear to build. Connectivity to Telegram, OpenRouter, and Google verified. The full project was then implemented:

- **Scaffold & dependencies** — `requirements.txt`, `.env.example`, `.gitignore`, package structure
- **Config, DB, encryption layer** — `app/config.py` (Pydantic settings + Fernet), `app/models.py` (Customer/Store/Job), `app/db.py` (async SQLAlchemy)
- **Google Sheets reader** — `app/sheets.py` (CSV export fetch, fuzzy AR+EN column matching)
- **LLM client** — `app/llm.py` (OpenRouter free model, graceful degrade if no key)

- **Shopify Admin API client** — `app/shopify.py` (REST Admin API 2024-10)
- **Salla API client** — `app/salla.py` (Admin API v1)
- **Job orchestration** — `app/jobs.py` (parse → enrich → push → QA report with progress callbacks)
- **FastAPI web dashboard** — `app/web.py` + templates (mobile-friendly, platform toggle, store creds management)
- **Telegram bot** — `app/bot.py` (aiogram 3, FSM conversational: asks platform → shop → token on first chat, reuses stored profile after)
- **Entry point** — `app/main.py` (runs uvicorn + bot together)

Verified via smoke tests

Test	Result
All 13 modules import cleanly	☑
Encryption roundtrip (Fernet)	☑
Sheet URL parsing + CSV parsing (AR+EN columns)	☑
DB init + models (customer/store/job)	☑
Web dashboard boots — <code>/healthz</code> 200, <code>/login</code> 200, POST login → 303, <code>/</code> 200	☑
LLM graceful degrade (no API key → uses raw data)	☑
Full job pipeline end-to-end (7/7 products published, progress streamed, QA report generated)	☑
Sample CSV parses correctly	☑

Decisions captured

- **Both Shopify & Salla** — toggle in the web dashboard, button choice in the bot ☑
- **Ask for API in each new chat unless in profile** — exactly the bot flow: first `/start` with no store → asks platform → shop → token, verifies, encrypts, stores. Next `/start` → "send me a sheet" ☑
- **Telegram first** ☑
- **Solo first** (single-customer; multi-tenant is roadmap) ☑
- **LLM:** OpenRouter free model wired (works with \$0); OpenHands Cloud is a documented drop-in swap (replace `enrich_product`). OpenRouter chosen because OpenHands Cloud is an automation runtime, not a free LLM endpoint ☑

4. Where to run the commands

User

where do i write the `cd /workspace/project ...` commands?

Assistant response

Important clarification: **the code only existed in the sandbox workspace**, which disappears when the session ends. To run it yourself, the files must first be on a real machine.

Two options:

- **Option A — Laptop (free, easiest to start):** get the files (push to GitHub, then `git clone`), open Terminal (Mac) / PowerShell (Windows) / terminal (Linux), `cd` into the cloned folder, run the commands there.
- **Option B — Small server (24/7 bot):** rent a \$5/mo VPS (DigitalOcean/Hetzner/Contabo) → SSH in → clone → run. The Telegram bot needs to be always-on.

What to get first (the "fill" part):

- `TELEGRAM_BOT_TOKEN` → `@BotFather` → `/newbot`
- `OPENROUTER_API_KEY` → sign up at openrouter.ai (free)
- `CREDENTIAL_ENCRYPTION_KEY` → `python -c "import secrets,base64;print(base64.urlsafe_b64encode(secrets.token_bytes(32)).decode())"`
- Python installed on the laptop

5. Making the repo

User

make the repo for now with good readme and documentation on how to use

Assistant action

- Wrote a comprehensive **README.md** (features, architecture, quick start, config table, sheet format with AR+EN columns, store-connection steps for Shopify & Salla, bot usage examples, web dashboard usage, deployment, cost, project structure, roadmap, troubleshooting, license)
- Added `docs/deployment.md` — Render/Railway + VPS (systemd + nginx + Let's Encrypt HTTPS) + backups
- Added `sample/products.csv` — a ready-to-test sample sheet (verified to parse)

- Added **LICENSE** (MIT)
- Committed everything: `9e2b30e` "Initial commit: Listing SaaS — Shopify/Salla product listing from Google Sheets"

Blocker encountered

The configured GitHub token had no `repo` scope, so the repository could not be created from the sandbox (403 "Resource not accessible by integration"). The token could read/push to existing repos, just not create new ones.

User asked to create the empty repo manually

The user was instructed to create an empty `listing-saas` repo at <https://github.com/new> (no README/.gitignore/license initialization, so the commit pushes cleanly), then reply "done".

6. Pushing to GitHub

User

done

Assistant action

- Verified the repo now exists (HTTP 200)
- Pushed using a token-embedded remote URL (the plain `git push` prompted for credentials interactively, which hangs in a non-interactive shell)
- Reset the remote URL afterward to avoid leaving the token in git config
- Verified: repo live, public, default branch `main`

Result — repo is live:

📄 <https://github.com/SoYama350/listing-saas> (public)

Contents pushed:

- All code (FastAPI dashboard + Telegram bot + Shopify/Salla/LLM/sheets clients)
- Full README with quick start, config, sheet format, store-connection, bot usage, troubleshooting
- `docs/deployment.md` — Render/Railway + VPS instructions
- `sample/products.csv` — ready-to-test sample sheet
- LICENSE (MIT)

Clone & run instructions provided

```
git clone https://github.com/SoYama350/listing-saas.git
cd listing-saas
pip install -r requirements.txt
cp .env.example .env
# fill in TELEGRAM_BOT_TOKEN, OPENROUTER_API_KEY, CREDENTIAL_ENCRYPTION_KEY
python -m app.main
```

Web → <http://localhost:8000> · Telegram → message your bot.

Note on visibility

The repo was created **public**. Offered to flip it to **private** via the API if the user prefers (recommended for a SaaS they may monetize, so the workflow/code isn't visible to competitors).

7. This documentation

User

can u upload the conversation in the repo and document it well in a pdf format or a markdown

Assistant action

This file ([docs/CONVERSATION.md](#)) is the markdown record of the full conversation. A PDF version ([docs/CONVERSATION.pdf](#)) is generated from it.

Project summary

Repo	https://github.com/SoYama350/listing-saas
What it does	Upload a Google Sheet of products → they go live on Shopify or Salla with auto-generated Arabic + English titles, descriptions, and tags
Interfaces	Web dashboard (laptop) + Telegram bot (phone)
Stack	FastAPI, aiogram 3, Shopify Admin API, Salla Admin API v1, OpenRouter LLM, SQLite, Fernet encryption
Cost to run	\$0 (free LLM model + free Telegram + laptop) to ~\$5/mo (VPS for 24/7)
Status	Built, smoke-tested end-to-end, pushed to GitHub

Roadmap (not yet built)

- WhatsApp channel (needs WhatsApp Business API / Twilio)
- Multi-tenant: clients self-serve (log in, paste their own sheet + store key)
- Billing (Stripe / Tap)
- OpenHands Cloud as the automation backend (drop-in replacement for `enrich_product`)
- CSV bulk export as a fallback to direct API

This document was generated as a record of the conversation that built the Listing SaaS project. Last updated: 2026-08-14.